

Week 3

Nadav Kohen

July 9, 2025

This week has a little less content volume than last week, so I want to encourage you to also review and become more comfortable with the Week 2 documents and their exercises as part of this week's assignment.

Now that we have a little bit of experience working with security definitions using attack games and probabilities, our goal this week is to make all of the details rigorous to pave the way for the study of the security of Schnorr digital signatures, which we will continue next week.

The primary missing piece in our formalism from last week is the definition of the notion that a probability is “negligible.” Recall that this notion was central to all of our security definitions, which were all the result of requiring all advantage values to be negligible.

For those who like accompanying visuals, see Figure 1.

Reading 1. *Read about negligible functions and probabilities in:*

Mike Rosulek - The Joy of Cryptography - pages 67-71.

In Reading 1, we encountered the following two equivalent definitions of what it means for a function, f , to be negligible:

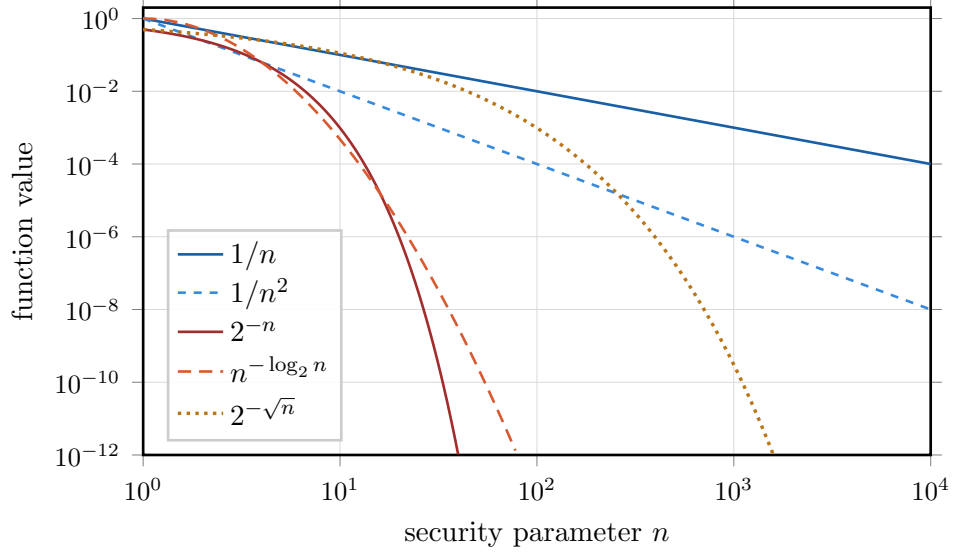


Figure 1: A negligible function eventually dips below every inverse polynomial, no matter how large its degree. On a log-log plot like the one above, the inverse polynomials $\frac{1}{n^c}$ are straight lines (with slope $-c$); a negligible function is a curve that eventually bends away from all such lines. Read the picture as a race to the bottom. The two blue lines are non-negligible reference functions – they decay, but only at a polynomial rate. Each warm curve is negligible, and each eventually undercuts every blue line.

- For all polynomials, p , $\lim_{\lambda \rightarrow \infty} p(\lambda) \cdot f(\lambda) = 0$.
- For all natural numbers, c , $\lim_{\lambda \rightarrow \infty} \lambda^c \cdot f(\lambda) = 0$.

The following are three additional equivalent definitions:

- For all polynomials, p , here exists a natural number, N , such that for all $\lambda > N$, $|f(\lambda)| \leq \frac{1}{|p(\lambda)|}$.
- For all polynomials, p , here exists a natural number, N , such that for all $\lambda > N$, $\frac{1}{|f(\lambda)|} \geq |p(\lambda)|$.
- For all natural numbers, c , here exists a natural number, N , such that for all $\lambda > N$, $\frac{1}{|f(\lambda)|} \geq |\lambda^c|$.

All three of these statements are more-or-less equivalent to the statement that f grows smaller faster than the reciprocal of any polynomial, or equivalently that the reciprocal of f

grows larger faster than any polynomial. From this perspective, I hope that it is clear that the first two definitions from the reading are also capturing this idea. With this intuition in mind,

Exercise 1. *Prove that a function, f satisfying any one of the above properties satisfies all of the above properties. (Some of you may prefer to do some or all of Exercise 2 before Exercise 1, which is totally fine).*

You may now use any of these definitions interchangeably as it suits you.

Exercise 2. *The final page of Reading 1 has exercises demonstrating properties of negligible functions. Complete Exercises 4.2, 4.3(a), 4.3(b), 4.4 and 4.5.*

Exercise 3. *Now that we have some more comfort with the formal definition of negligible, double-check, for rigor, your solutions to Exercises 4 from 2a and 8 from last week (2b).*

One last note on our formal model for cryptographic security proofs is the type of adversaries we allow. Namely, we do not allow the adversary to be an arbitrary program, but rather we require it to be a Probabilistic Polynomial Time (PPT) program. By “Probabilistic,” we mean that the adversary is allowed to use randomness, such as internal coin flips. By “Polynomial Time,” we mean that the number of steps in any execution of $\mathcal{A}$ must be bounded by some polynomial, $p(\lambda)$. This assumption is not only sensible and practical, but it is also necessary for our definition of negligible to work because it ensures that our adversary cannot turn a negligible success probability into a non-negligible one by brute force (for example, the adversary may not sample a non-negligible proportion of all possible private keys).

Let us now begin our study of Schnorr signatures, beginning with the definition of security of a signature scheme: (keep in mind that an “oracle” in cryptography is simply a program that all other programs have access to as an interface that takes an input and returns an output, but no party has access to any information about the source code of this program

beyond the specification; also note that a PPT adversary can make at most a polynomial-bounded number of queries to any oracle)

Reading 2. *Read about the definition of Existentially UnForgable under Chosen Message Attack (EUF-CMA) in:*

Jonathan Katz and Yehuda Lindell - Introduction to Modern Cryptography - pages 463-468.

Note that when Katz and Lindell refer to n , they are talking about the security parameter, which we have been calling λ .

Before practicing with this definition, let us restate the book’s definition in the attack-game notation we used last week, and then work through two examples — one proving a scheme secure and one breaking a scheme — since the exercise below will ask you to do both.

A (digital) signature scheme is a triple of PPT algorithms (G, S, V) : the key generation algorithm $G(\lambda)$ outputs a key pair (pk, sk) ; the signing algorithm $S(sk, m)$ outputs a signature σ on a message m ; and the (deterministic) verification algorithm $V(pk, m, \sigma)$ outputs true or false. We require correctness: an honestly produced signature always verifies. The EUF-CMA attack game from the reading can then be written as the game $\text{EUF-CMA}^A(\lambda)$ in Figure 2. The adversary is given the public key and access to a signing oracle SIGN (modeling the “chosen message attack”), and it wins if it can produce a valid signature on *any* message it did not already ask the oracle to sign (modeling “existential forgery”). The set Q simply records every message the adversary has queried, so that the final check $m^* \notin Q$ enforces that the forgery is on a fresh message.

As usual, we define the advantage of an adversary to be its probability of winning,

$$\text{Adv}_{\mathcal{A}}^{\text{euf-cma}}(\lambda) := \Pr [\text{EUF-CMA}^A(\lambda) = \text{true}] ,$$

and we say that the signature scheme (G, S, V) is secure (i.e., EUF-CMA secure) if $\text{Adv}_{\mathcal{A}}^{\text{euf-cma}}(\lambda)$

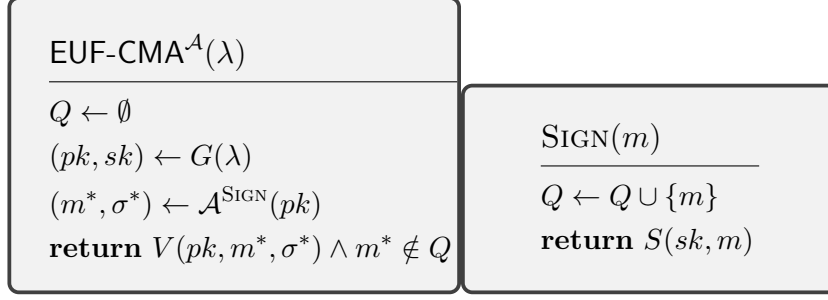


Figure 2: The EUF-CMA attack game for a signature scheme (G, S, V) . The adversary $\mathcal{A}$ has oracle access to SIGN , which signs any requested message and records it in Q ; $\mathcal{A}$ wins by outputting a valid signature on a message it never sent to SIGN .

is negligible for every PPT adversary $\mathcal{A}$. Notice that, unlike the distinguishing games from last week, here there is no “ $-\frac{1}{2}$ ”: a forger does not get to win half the time by guessing, since blindly outputting a signature on a fresh message succeeds only with negligible probability against a secure scheme.

Example: proving security by reduction. Suppose (G, S, V) is an EUF-CMA secure signature scheme with message space $\{0, 1\}^\lambda$. Consider the new scheme (G, S', V') with message space $\{0, 1\}^{\lambda-1}$ that signs a message after prepending a fixed 0 bit:

$$S'(sk, m) = S(sk, 0 \parallel m), \quad V'(pk, m, \sigma) = V(pk, 0 \parallel m, \sigma),$$

where $\parallel$ denotes concatenation and key generation is unchanged. Intuitively, one would assume that (G, S', V') is also EUF-CMA secure since adding this zero bit is inconsequential, but there are many situations in cryptography when seemingly inconsequential alterations do break the security of a protocol. As such, we will rigorously prove that this modification is secure if you started with a secure signature scheme, and of course we will prove this by reduction.

Following the recipe from Week 2a, the implication we want is “ $((G, S, V)$ is secure) $\Rightarrow ((G, S', V')$ is secure),” so we prove its contrapositive: we assume a PPT adversary $\mathcal{A}$ that

forges against (G, S', V') with non-negligible advantage, and we build a PPT adversary $\mathcal{A}'$ that forges against (G, S, V) with non-negligible advantage. Our “magic box” is $\mathcal{A}$; the new wrinkle, compared to the reductions of Week 2a, is that $\mathcal{A}$ expects to interact with a signing oracle, so $\mathcal{A}'$ must *simulate that oracle* for $\mathcal{A}$.

The reduction $\mathcal{A}'$ is given pk and access to the signing oracle $\text{SIGN}(\cdot) = S(sk, \cdot)$ for the base scheme, and works as follows (see Figure 3 for the corresponding box-within-a-box diagram).

- (1) Run $\mathcal{A}(pk)$.
- (2) Whenever $\mathcal{A}$ queries its signing oracle on a message $m \in \{0,1\}^{\lambda-1}$, answer it by querying our own oracle on $0 \parallel m$ and returning the result $S(sk, 0 \parallel m)$. Since this equals $S'(sk, m)$, from $\mathcal{A}$ ’s point of view it is talking to a genuine (G, S', V') signing oracle.
- (3) When $\mathcal{A}$ outputs a forgery (m^*, σ^*) , output $(0 \parallel m^*, \sigma^*)$.

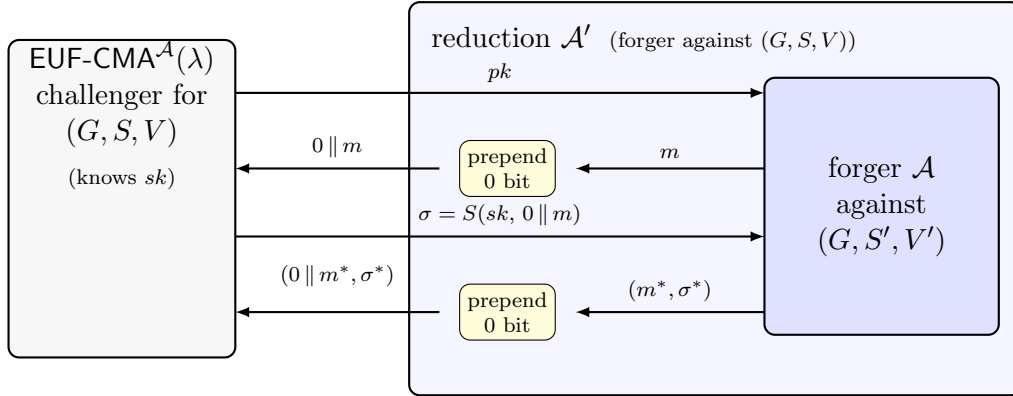


Figure 3: The security reduction drawn in the standard “box-within-a-box” style.

Now we check that $\mathcal{A}'$ wins its game exactly when $\mathcal{A}$ wins. First, whenever $\mathcal{A}$ ’s forgery verifies, so does ours, because $V'(pk, m^*, \sigma^*) = V(pk, 0 \parallel m^*, \sigma^*)$ by definition. Second, we must confirm the forgery is on a fresh message, i.e., that $0 \parallel m^*$ was never sent to our own

oracle. The only messages we ever sent to our oracle were of the form $0 \parallel m$ for messages m that $\mathcal{A}$ queried, and $\mathcal{A}$ winning means m^* differs from all of those m ; hence $0 \parallel m^*$ differs from every message we queried, so it is fresh. Therefore $\mathcal{A}'$ wins precisely when $\mathcal{A}$ does, giving $\text{Adv}_{\mathcal{A}'}^{\text{euf-cma}}(\lambda) = \text{Adv}_{\mathcal{A}}^{\text{euf-cma}}(\lambda)$, which is non-negligible. Finally, $\mathcal{A}'$ is PPT because it just runs $\mathcal{A}$ (which is PPT) plus one extra bit of bookkeeping per oracle query. This contradicts the assumed security of (G, S, V) , completing the reduction.

It is worth naming the moving parts, since every EUF-CMA security proof you write will have them: (i) identify which scheme's security is the assumption and which is the conclusion; (ii) use the forger for the conclusion-scheme as a subroutine; (iii) *simulate its signing oracle* using your own oracle; (iv) *translate its forgery* into a forgery for your scheme; and (v) verify that your translated forgery is both valid *and* on a message you never queried.

Example: breaking a scheme. Not every transformation preserves security, and showing that a scheme is *insecure* is usually easier: we only have to exhibit a single PPT adversary with non-negligible advantage. Consider the scheme (G, S', V') on message space $\{0, 1\}^\lambda$ (with $\lambda \geq 2$) that signs the message after *deleting its last bit*: writing $m = b_1 b_2 \cdots b_\lambda$,

$$S'(sk, m) = S(sk, b_1 \cdots b_{\lambda-1}), \quad V'(pk, m, \sigma) = V(pk, b_1 \cdots b_{\lambda-1}, \sigma).$$

The problem is that any two messages differing only in their last bit receive the very same signature, which lets us forge after a single query (see Figure 4):

- (1) Query the signing oracle on $m = 0^\lambda$ (the all-zeros message), receiving $\sigma = S'(sk, 0^\lambda) = S(sk, 0^{\lambda-1})$.
- (2) Output the forgery $(m^*, \sigma^*) = (0^{\lambda-1}1, \sigma)$.

The message $m^* = 0^{\lambda-1}1$ was never queried (it differs from 0^λ in the last bit), so it is fresh, yet $V'(pk, m^*, \sigma) = V(pk, 0^{\lambda-1}, \sigma) = \text{true}$ because σ is a genuine base-scheme signature

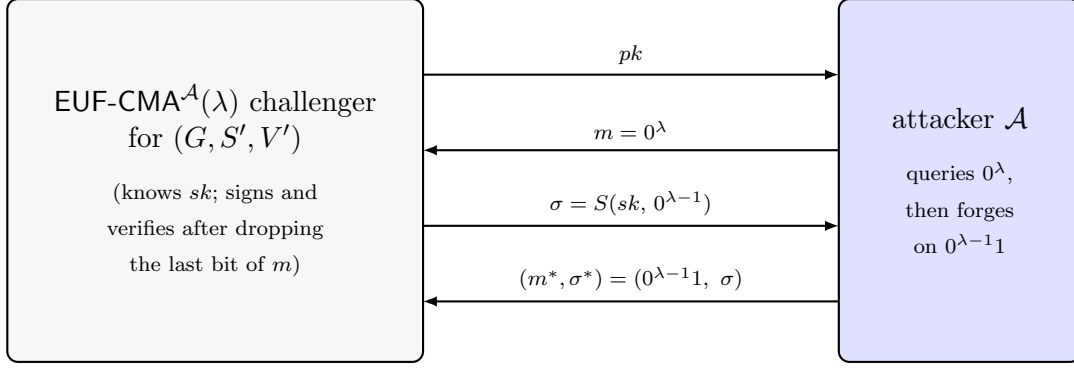


Figure 4: The forgery attack of the previous example. Unlike Figure 3, this is not a reduction: the attacker $\mathcal{A}$ is a single program playing the EUF-CMA game directly against the modified scheme (G, S', V') , whose signing oracle and verifier both drop the last bit of a message.

on $0^{\lambda-1}$. This adversary wins with probability 1, so $\text{Adv}_{\mathcal{A}}^{\text{euf-cma}}(\lambda) = 1$, which is certainly not negligible, and hence (G, S', V') is not EUF-CMA secure. Notice the shape of an attack: choose specific oracle queries, output a specific forgery, and argue both that it verifies and that its message is fresh.

Now you are ready to try both directions yourself.

Exercise 4. Let (G, S, V) be a secure signature scheme with the message space $\{0, 1\}^\lambda$. Let $\hat{G}$ be a new key generation function that simply calls G twice and returns a pair of secret keys, (sk_0, sk_1) , and the corresponding pair of public keys, (pk_0, pk_1) . Which of the following signature schemes, $(\hat{G}, \hat{S}, \hat{V})$, are secure? Show an attack or prove security (by reduction).

(a) Prove that “Sign halves” is **not** secure:

$$\hat{S}((sk_0, sk_1), (m_L, m_R)) = (S(sk_0, m_L), S(sk_1, m_R)),$$

$$\hat{V}((pk_0, pk_1), (m_L, m_R), (\sigma_0, \sigma_1)) = V(pk_0, m_L, \sigma_0) \wedge V(pk_1, m_R, \sigma_1).$$

(b) Show that “Sign with randomness” is **not** secure:

$$\hat{S}(sk_0, m) = [\text{choose random } r \in \{0, 1\}^\lambda; \text{output } (r, S(sk_0, m \oplus r), S(sk_0, r))],$$

$$\hat{V}(pk_0, m, (r, \sigma_0, \sigma_1)) = V(pk_0, m \oplus r, \sigma_0) \wedge V(pk_0, r, \sigma_1).$$

(c) Prove that “Accept one valid” is secure:

$$\hat{S}((sk_0, sk_1), m) = (S(sk_0, m), S(sk_1, m)),$$

$$\hat{V}((pk_0, pk_1), m, (\sigma_0, \sigma_1)) = V(pk_0, m, \sigma_0) \vee V(pk_1, m, \sigma_1).$$

(Hint: Consider using that $Pr[\sigma_0 \text{ or } \sigma_1 \text{ is valid}] \leq Pr[\sigma_0 \text{ valid}] + Pr[\sigma_1 \text{ valid}]$ from Exercise 4 of last week).

Now we are almost ready to see the Fiat-Shamir Transform (next week), which turns secure identification protocols into secure digital signature schemes. We will use the Fiat-Shamir Transform to prove the security of the Schnorr signature soon. The only ingredient missing is the Random Oracle Model (ROM), which is how we model our hash functions to make proofs more practical:

Reading 3. Read about the Random Oracle Model in:

Jonathan Katz and Yehuda Lindell - Introduction to Modern Cryptography - pages 187-191.

Pieter recorded the following video further discussing the ROM: <https://www.youtube.com/watch?v=ITxL4b0k0y0>

One strange but important feature of the ROM discussed in the section “Definitions and Proofs in the ROM” is that during a reduction, adversaries $\mathcal{A}$ must “report” all of their oracle queries to the adversary $\mathcal{A}'$ that is using $\mathcal{A}$ as a subroutine, and additionally $\mathcal{A}'$ may adaptively control the response that $\mathcal{A}$ gets to these queries so long as $\mathcal{A}$ cannot distinguish what they receive from a regular random oracle. This kind of reporting feature is also key to other models we will discuss in the future.

Next week, we will use the Random Oracle Model (ROM) to construct the Fiat-Shamir transform, and then we will dive straight into proving that the Schnorr digital signature is secure!